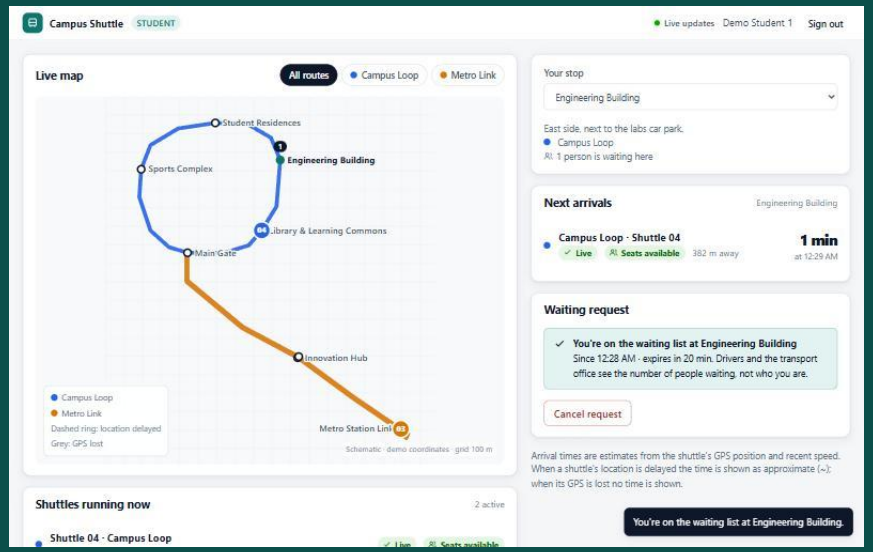
# Campus Shuttle Tracking & Demand Prediction

**Problem** Students cannot see where the campus shuttle is, so they wait outdoors or miss it; the transport office plans the fleet without data.

**Purpose** Live positions and honest arrival estimates — honest above all when GPS or mobile data fails — and "I'm waiting here" requests that become demand history and explainable forecasts.

**Abdullah shammari · Zuhair Hussein Halawa · Mubarak Al Khyeli · Ali Yosuf Ameer**

BCS 305 Software Architecture · Dr. Khaled Ammar · Fall 2026




Student view of the running prototype (demonstration data)

# Three user groups, five use cases, one hard quality

## Stakeholders

**Students** — where is the shuttle, when will it come?
**Drivers** — one-tap trips; not blamed for GPS faults
**Transport office** — live fleet, incidents, demand to plan with

Also affected: university IT (runs it), the data-protection officer (driver locations and student identities are personal data), management (funds the fleet).

## Primary use cases

- UC1 Track shuttles and view arrival times
- UC2 Run a trip (driver; GPS receiver)
- UC3 Register a waiting request
- UC4 Monitor fleet and waiting demand; end a trip remotely
- UC5 View and explain a demand forecast

**Demonstrated end to end: UC2 → UC1, and UC3 → UC4 → UC5**

## Quality scenarios (measured)

- Freshness: new position to every viewer, p95 ≤ 1 s
- GPS outage: "delayed" after 20 s, "lost" after 60 s, no estimate when lost
- Network loss: positions queued and delivered in order (bounded: 10 min, newest 500)
- Security: 26 endpoints × 4 caller types checked
- Privacy: no student location collected

**Hard quality attribute** timely and honest shuttle information under unreliable GPS and mobile connectivity: fast when the data is good, and saying so — instead of guessing — when it is not.

# Process: incremental, risk first, then reviewed iterations

## What we actually did (Module 02 terms)

**Incremental** — new parts in each slice: skeleton → tracking → driver and student pages → demand → hardening (baseline); a 4th forecaster (#12); browser tests and CI (#13)

**Spiral idea** — four unknowns prototyped first: SSE fan-out (load test), GPS states, route snapping, ML feasibility

**Iterative** — the same parts improved in reviewed rounds: #14–#17 and this submission pass, each through a pull request with approval and a passing check

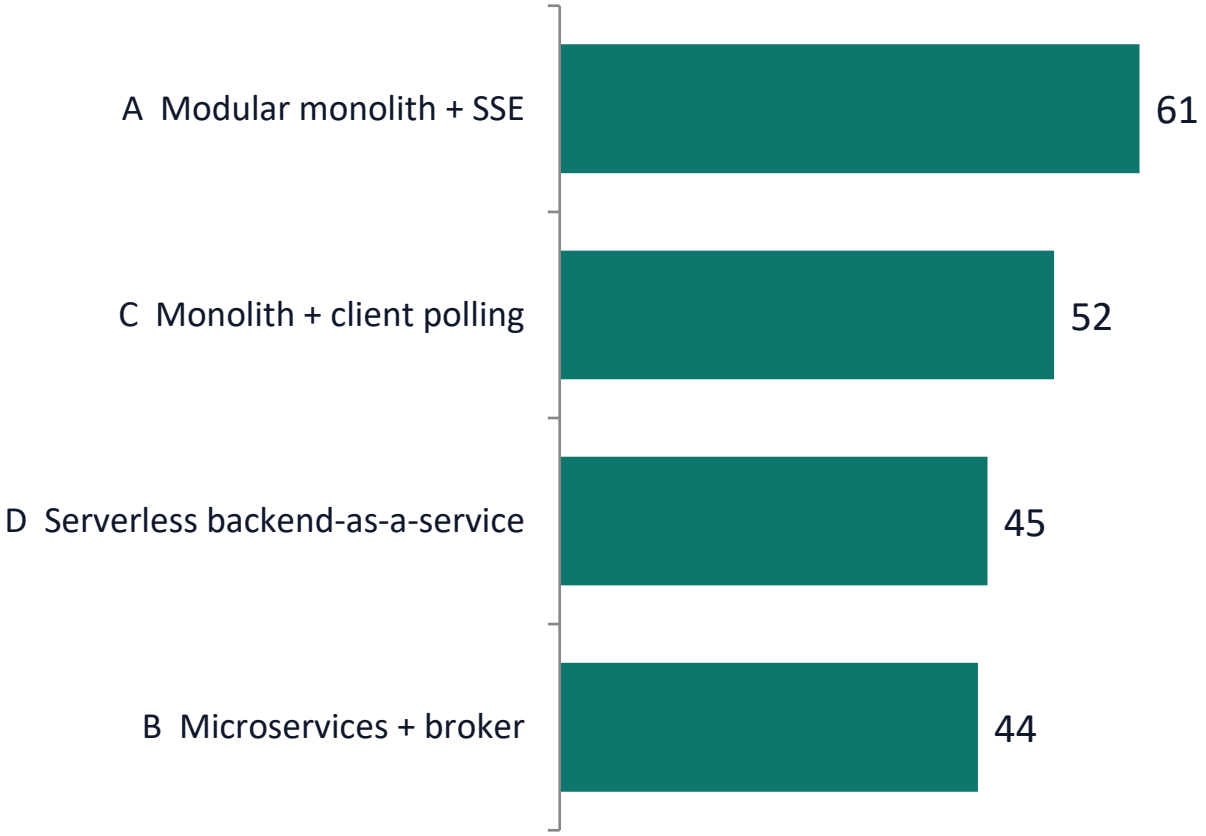
V-model pairing for tests. Not used: Scrum sprints, waterfall sign-off.

**Lesson:** the recorded work is compressed into 31 Oct – 10 Nov. Starting the repository at the proposal would have left time for stakeholders and a field test (report §9).

| | | |
|---|---|---|
| ○ | **Wk 3** | Team studio, topic proposal<br>course plan |
| ○ | **Wk 4** | Topic approval; proposal due<br>date not recorded |
| ● | **31 Oct** | Baseline: 5 slices, load tests<br>repository |
| ● | **1–2 Nov** | PRs #12–#16, reviews, protected main<br>GitHub |
| ● | **6–9 Nov** | PR #17: review, 4 fixes, approval, merge<br>GitHub |
| ● | **10 Nov** | Submission pass: SSD, UML print, video<br>this branch |
| ○ | **Wk 14–15** | Presentation and demo<br>slot not yet published |

Filled dot: in the repository record · open dot: course schedule (indicative) — our own dates for these are not recorded

# Four architectures compared — the modular monolith wins




- Weighted against freshness, robustness, security & privacy, modifiability, team-of-two simplicity, growth
- A: best measured freshness, one trust boundary, one command to run
- C: equal to A except freshness — 1 s polling means 1,000–3,000 requests/s
- D: location data held by a third party; GPS timeouts need a scheduler
- B: independent scaling one campus does not need; ≥ 5 deployables

Scores are a technical judgement (report §6.3); only A's freshness is measured.

# One process, eight components, enforced boundaries

**Student UI** | **Driver UI + GPS simulator** | **Office UI**

Browsers: REST (JSON) + Server-Sent Events over HTTP

**Campus Shuttle server — one Node.js process in TypeScript (src/)**

**Web API — auth, role checks, CSRF, CSP (web/)**

**Real-time Gateway — SSE, coalescing (realtime/)**

**Identity & Access (identity/)**

**Network & routes (network/)**

**Tracking, ETA, GPS states (tracking/)**

**Demand & forecasting (demand/)**

**Event Bus — domain events, Observer (shared/)**

**Persistence — SQLite repositories (persistence/)**

test/architecture.test.ts fails if an import breaks these rules (imports only, not every UML detail)

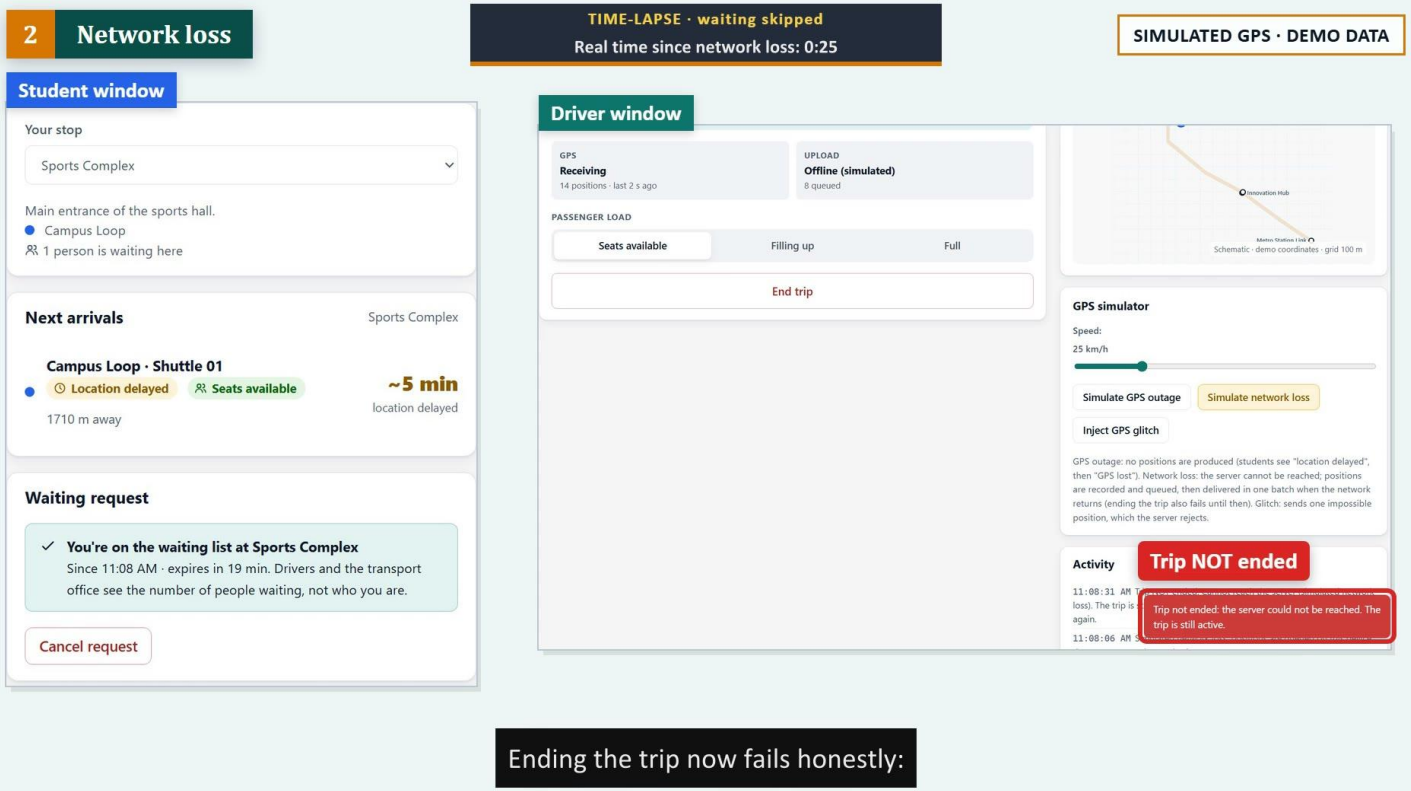
**Deployment** one server runs the Node.js process and one SQLite database file; phones and laptops use a browser. No broker, no cloud service. HTTPS behind a reverse proxy is the next production step (UML A.12).

# Five patterns, each tied to behaviour you will see

| Pattern | Behaviour in the prototype | Where in the code |
|---|---|---|
| **State** | Live → Location delayed (20 s) → GPS lost (60 s) → Live; what passengers may be told | `tracking/signal-state.ts — one class per state` |
| **Adapter** | The demo drives the same upload path as a real phone | `public/js/gps.js — simulated and device GPS, one contract` |
| **Observer** | One upload reaches every browser and the waiting module | `shared/event-bus.ts → SseHub, WaitingService` |
| **Repository** | Positions stored before memory changes; tests run without a database | `persistence/sqlite-*.ts, in-memory doubles` |
| **Strategy** | Arrival-time and forecasting models swapped and compared | `EtaStrategy, DemandPredictor (4 models)` |

Class diagrams of all five: UML set A.13–A.17. One composition root (src/container.ts) wires them; tests swap the clock, the database and the strategies.

# Demonstration: what happens when connectivity fails?




Recorded demonstration, 4:11: supplied separately as 06-Demo.mp4 with 06-Demo.srt

## In the video and live

- Normal trip: driver starts, student sees Live, office sees the same trip
- Network loss: positions queued; "Trip NOT ended"; 10 positions delivered in one batch
- GPS outage: Live → Location delayed → GPS lost, no estimate → Live
- Waiting request → office count → served at the stop
- Forecast cell → how the number was calculated (synthetic history)

Real prototype, three browser windows, throw-away demo database. Simulated GPS; skipped waiting time is labelled TIME-LAPSE with the real elapsed time. Narration: synthetic voice.

# Verification, limitations and what we learned

**144**

119 Node + 25 Chromium tests, all passing (incl. accessibility)

**26 × 4**

endpoints × caller types checked for role access

**35–37 ms**

p95 to 1,000 simulated viewers — local benchmark, one PC

**21**

defects and corrections in the test plan, all resolved

## Limitations

- GPS behaviour tested with the simulator only — no real phone or vehicle yet
- A driver page in a background tab stops reporting; a native app may be needed
- Forecasts shown on synthetic history: the method, not accuracy
- No HTTPS, university single sign-on, Arabic or screen-reader session yet; the benchmark is not a deployment test

## Reflection (report §9)

- Spread the work: the record starts in Week 9
- Talk to stakeholders before fixing numbers such as 1 s, 20 s and 60 s
- Check content, not existence: an independent reviewer found what the author's own checks missed
- Keep claims at the strength of the evidence

Each member confirms or corrects this reflection in person.

# Conclusion

A modular monolith with server push gives fresh positions in milliseconds locally — and, more importantly, says "delayed" and "lost" instead of guessing when GPS or the network fails.

Next: a field test with real phones, HTTPS and single sign-on, then real demand data before any trained model.

### Contributions (factual record, report §10)

**Abdullah shammari:** repository and protected workflow; integration pull requests #14–#18; merges

**Zuhair Hussein Halawa:** forecasting model (#12); simulator, browser and accessibility tests (#13); reviews and approvals of #14–#18; README media

**Mubarak Al Khyeli:** first-run check of the setup instructions

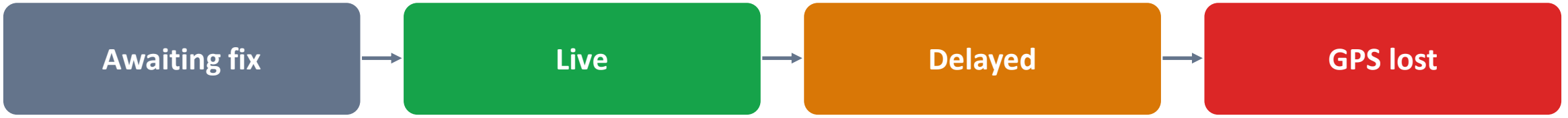
**Ali Yosuf Ameer:** worked forecast example, checked against the dashboard

AI tools and personal statements: report §10

# Questions?

Backup slides follow: GPS states and network loss · synthetic backtest · one upload, measured

# Backup · When GPS fails, the system says so instead of guessing



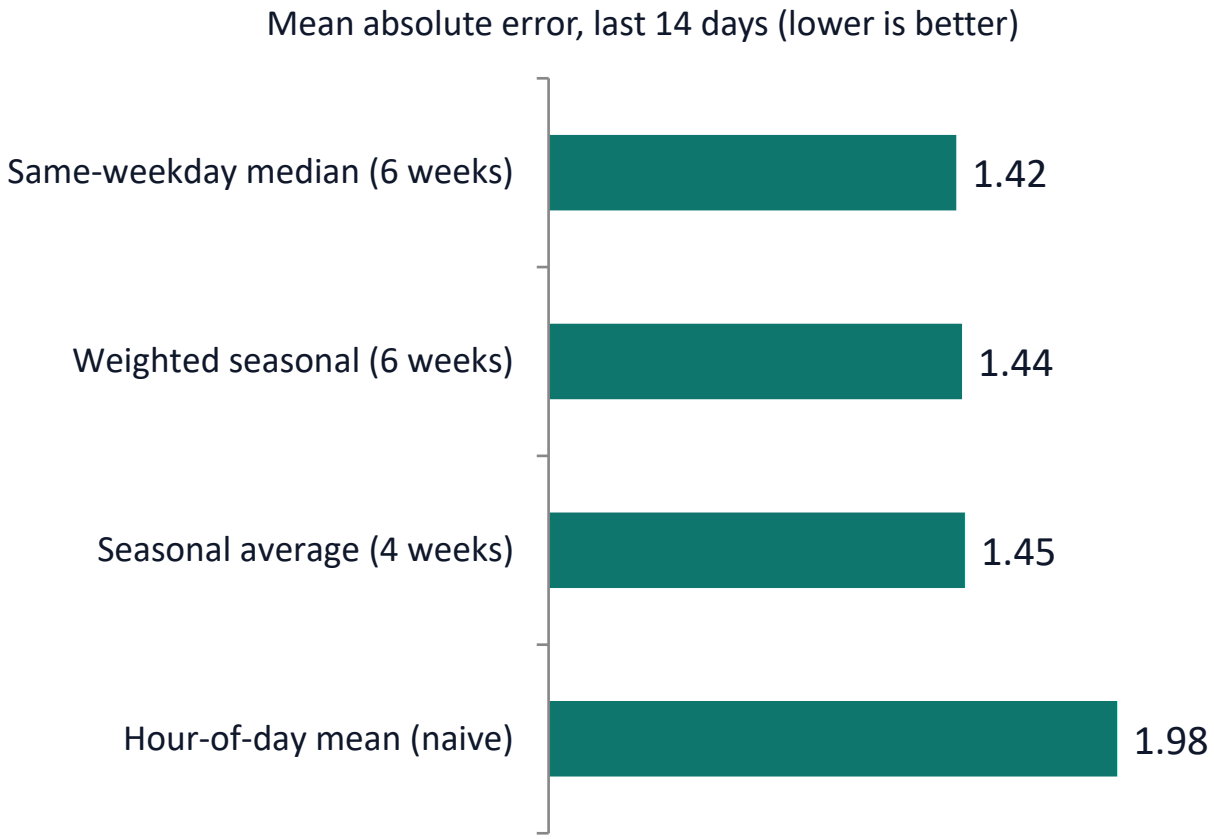

| Awaiting fix | Live | Delayed | GPS lost |
|---|---|---|---|
| trip started, no position yet | newest position ≤ 20 s old | 20–60 s without a position | more than 60 s |
| **Students see** "waiting for first GPS position" | **Students see** ETA: "4 min · at 08:14" | **Students see** ETA marked "~4 min", then "Due" | **Students see** no ETA — "last seen 2 min ago" |

**Any accepted position → Live again · no position for 30 min → trip closed automatically**

## Network loss on the driver's phone

Positions are queued in the open driver page (newest 500 kept) and retried with backoff (1 → 15 s). On reconnection the server corrects the phone's clock using sentAt, sorts them, and rejects duplicates, positions over 10 min old and impossible jumps. Ending the trip fails honestly while offline. Closing the page loses the queue.

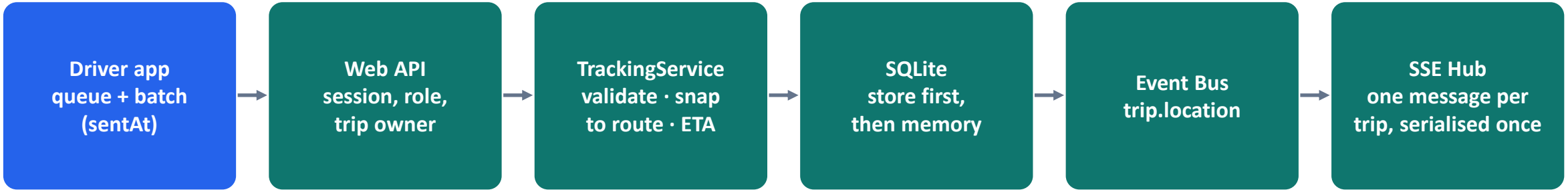
# Backup · Synthetic backtest: the method works, accuracy unproven




## SYNTHETIC DATA

- History generated from our own written assumptions (weekday peaks, quiet weekends) — not from any university
- Weekday-aware models beat the naive one because the data was built that way: this checks the pipeline, nothing more
- With one semester of real requests, the same backtest decides whether a trained model is worth adding (RBC Borealis lifecycle: feasibility first)

# Backup · One GPS upload reaches every viewer in milliseconds




| Local benchmark (1 PC, Node 24) | p50 | p95 | max | skipped |
|---|---|---|---|---|
| 1,000 viewers × 60 updates | 23–25 ms | 35–37 ms | 55–56 ms | 0 |
| 3,000 viewers × 30 updates | 70–72 ms | 90–98 ms | 108–113 ms | 0 |

npm run load-test · viewers share the server process over loopback, one session, no browsers: a fan-out benchmark with simulated viewers, not a field test (31 Oct and 1 Nov runs)

- Server-Sent Events: browsers reconnect by themselves and get a full snapshot
- A slow phone skips updates and is resynchronised — it cannot grow server memory
- Positions are stored before memory changes: a restart loses nothing